

Flow Matching as a Layer

Learning to reshape frozen image features for a classifier that never learns again

Yousef Shihade · Mira Bitar | University of Haifa · 2026

This project asks a single question in three stages. Given a frozen pretrained encoder and a fixed classification rule, can a small flow-matching network placed between them learn to transport features to where the classifier wants them, and when does that help? Stage 1 established the ground truth: a trained linear probe and an untrained prototype classifier on two frozen encoders and two datasets, evaluated under one protocol. Stage 2 put a flow in front of the prototype classifier and found gains of up to +22.3 pp, with the size of the gain predicted exactly by how much a trained classifier had beaten the prototypes in Stage 1. Stage 3 put a flow in front of the already-trained linear probe and found the gain shrinks to +0.1 to +2.6 pp, which is what Stage 2's result predicts once that headroom has been spent. Along the way the three stages agree on a second lesson: what a classifier consumes is its decision margin, and neither cluster tightness nor distance to a prototype is a substitute for it. All findings rest on 306 trained models, three seeds per cell, the full official test splits, and bit-for-bit reproduction of every earlier stage.

67.55%

Stage 1: best baseline, the DINOv2 linear probe on Aircraft. A 10× separability margin measured before any training predicted the encoder ordering.

+22.3 pp

Stage 2: a flow in front of untrained prototypes. Gain tracks Stage 1 headroom with Spearman $\rho = 1.000$; the flow recovers about 70% of it.

4 of 18

Stage 2: rolled-out training loses to standard FM. It transports better and decides worse, raising similarity to rival prototypes just as much.

+0.1–2.6 pp

Stage 3: a flow in front of the trained probe. Modest, as the headroom rule predicts; unfreezing the classifier changes nothing (-0.02 pp).

1 The question and the shared protocol

Every experiment in the project runs on the same pipeline. An image passes through a frozen pretrained encoder to give a feature z ; a classification rule maps z to a label. Stage 1 measures how good that rule can be on its own. Stages 2 and 3 insert one trainable component, a velocity network $v(z, t)$ integrated for T Euler steps, between the two, and hold everything else fixed:

$$\text{image} \rightarrow [\text{frozen encoder}] \rightarrow z \rightarrow [\text{FM layer, } T \text{ Euler steps}] \rightarrow \hat{z} \rightarrow [\text{fixed rule}] \rightarrow \text{label}$$

The two later stages differ only in which rule sits on the right and where its targets come from. In Stage 2 the rule is the untrained prototype classifier and the flow is trained toward class prototypes. In Stage 3 the rule is the trained linear probe, frozen, and the flow is trained to make that probe’s decisions better. Because the encoder, the cached features, the k-shot subsets, the seeds and the test splits never change, any accuracy difference between a stage and its baseline is attributable to the flow alone.

Datasets	DTD (47 texture classes, official partition 1) and FGVC-Aircraft (100 variant-level classes); official train/val/test splits kept separate, never merged
Frozen encoders	ResNet-18 (torchvision IMAGENET1K_V1, 512-d, both datasets); DINOv2 ViT-S/14 (384-d, Aircraft only). Three encoder/dataset combinations throughout
Training sizes	$K \in \{5, 10, \text{full}\}$. 5- and 10-shot use 3 balanced random subsets (seeds 0/1/2); full uses the complete training split with 3 seeds. Stage 3 fixes $K = 10$
Metric	Top-1 accuracy on the complete, untouched official test split (1880 / 3333 / 3333), mean $\pm$ sample std over 3 seeds
Velocity network	MLP, 2×512 hidden, SiLU, t concatenated to the input; carried unchanged from Stage 2 into Stage 3, never tuned. Euler integration with $T = 4$ (Stages 2 and 3) and $T = 12$ (Stage 2)

Two properties of the cached features recur in every stage and are worth stating once. First, the encoders’ class structure is very uneven: the cosine-similarity margin between a test feature’s own class centroid and the other centroids is 0.172 for ResNet-18 on DTD, only 0.041 for ResNet-18 on Aircraft, and 0.420 for DINOv2 on Aircraft. Second, feature norms sit around 24 to 50 depending on the encoder and are never near 1; ResNet-18 features are non-negative and DINOv2 features are signed. The first fact predicts the accuracy ordering in every stage; the second decides, differently in Stages 2 and 3, whether features must be normalised before the flow sees them.

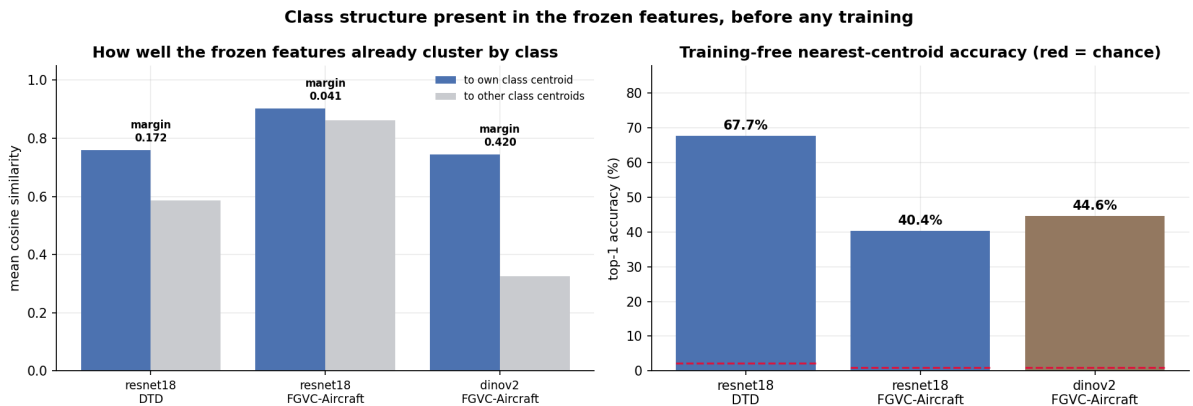


Figure 1. Class structure present in the frozen features before any classifier exists (Stage 1). Left: mean cosine similarity to own versus other class centroids. Right: training-free nearest-centroid accuracy. ResNet-18 on Aircraft is the hardest case by an order of magnitude.

2 Stage 1: frozen-encoder baselines STAGE 1

Stage 1 built the two classifiers that Stages 2 and 3 later work against, and made sure both were trustworthy. Before any feature was extracted, twenty automated checks per dataset confirmed the official partitions, split sizes, pairwise disjointness and file presence. Both encoders were then run once over every image (25,640 in total), with the checkpoint’s own preprocessing (resize 256, centre-crop 224) kept unchanged even though it discards about 41% of each Aircraft image’s width; deviating from the published preprocessing would have broken comparability with the frozen encoder and with the later stages. A 20-pixel copyright banner printed into every Aircraft image was cropped beforehand, since it is a source-image artifact rather than part of the photograph, and a direct ablation showed the crop changes accuracy by 0.78 pp, inside the ± 1.55 pp noise margin.

The linear probe, $s = Wz + b$, was trained with AdamW (lr 10^{-3} , weight decay 10^{-4} , batch 64, up to 200 epochs, checkpoint on highest validation accuracy) across 27 runs; no run diverged and every run selected a real checkpoint. The prototype classifier trains nothing: it averages each class’s L2-normalised training features into one vector and classifies by cosine similarity. We chose prototypes over zero-shot CLIP as the second baseline because they reuse the same cached features and the same k-shot subsets as the probe, which makes the comparison between the two a statement about classifier design rather than training data.

Dataset / Encoder	K	Linear probe	Prototype	Head-room
DTD / ResNet-18	5	44.86 ± 1.83	46.10 ± 1.89	−1.24
	10	52.04 ± 1.16	51.67 ± 1.33	+0.37
	full	62.82 ± 0.42	58.83 (1 run)	+3.99
Aircraft / ResNet-18	5	20.76 ± 0.26	16.46 ± 0.39	+4.30
	10	27.87 ± 1.33	20.19 ± 1.14	+7.68
	full	38.15 ± 0.29	25.47 (1 run)	+12.68
Aircraft / DINOv2	5	38.19 ± 2.49	23.79 ± 0.47	+14.40
	10	51.54 ± 1.70	27.74 ± 0.98	+23.80
	full	67.55 ± 0.28	34.26 (1 run)	+33.29

Table 1. Stage 1 top-1 test accuracy (%) for every dataset, encoder, K and baseline. The full-data prototype is deterministic and so has a single run. Headroom, the probe’s gain over the prototype on the same cell, is the quantity Stage 2 turns out to hinge on.

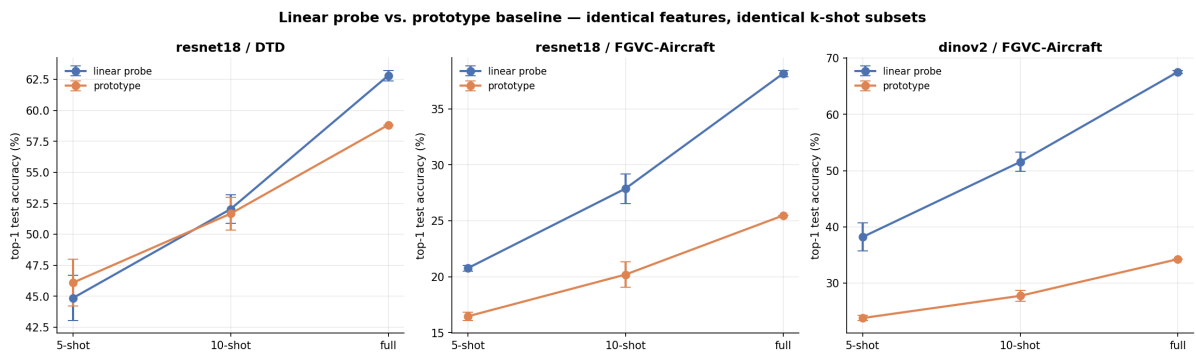


Figure 2. Linear probe versus prototypes on identical features and identical k-shot subsets, error bars over 3 seeds. Close on DTD, where prototypes even edge ahead at 5-shot; on Aircraft the probe pulls away as K grows, by up to 33 pp at full data.

Three findings carry forward. The linear probe wins everywhere, and by a widening margin on Aircraft, because averaging a class into one vector destroys fine-grained detail that a trained boundary can still use; on ResNet-

18/Aircraft, 85 of the 100 classes score below 50% own-prototype accuracy even though the aggregate is 25.4%. DINOv2 substantially outperforms ResNet-18 on Aircraft (67.55% against 38.15% at full data), consistent with the tenfold wider separability margin in Figure 1. And the confusions are the ones a sensible model should make: on DTD they sit among related pattern concepts (dotted $\leftrightarrow$ polka-dotted, lined $\rightarrow$ banded), and on Aircraft inside manufacturer families (C-47 $\leftrightarrow$ DC-3, A340-200 $\rightarrow$ A340-300). Training behaviour also differed by dataset in a way that recurs later: DTD probes peak around epoch 18–32 and then overfit, while Aircraft probes are still improving past epoch 150, tracking the separability gap; tripling the epoch budget changed accuracy by at most 0.21 pp.

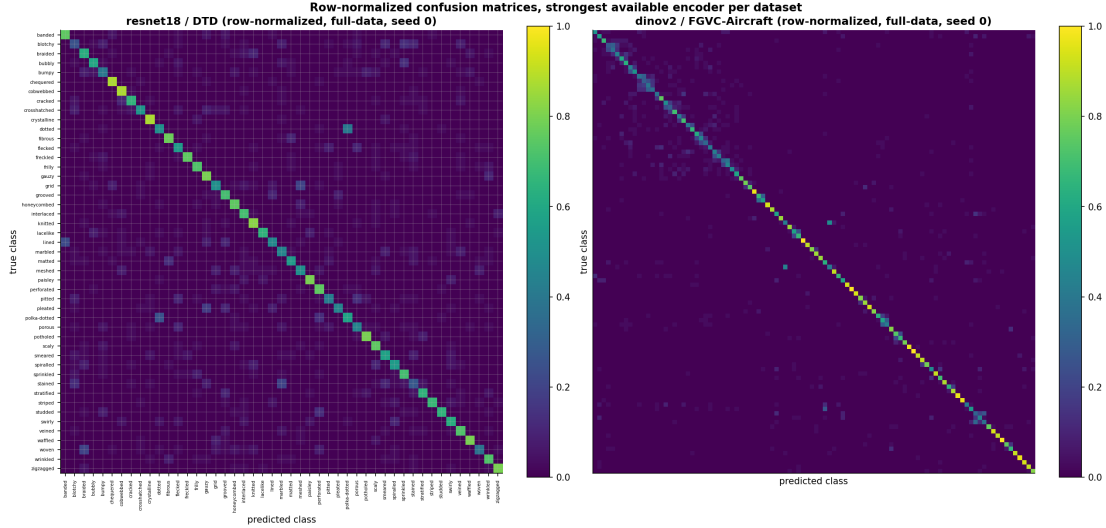


Figure 3. Row-normalised confusion matrices for the strongest encoder on each dataset (full data, seed 0). DTD’s 47 classes are labelled; Aircraft’s 100 are not, for readability.

3 Stage 2: flow matching to class prototypes STAGE 2

Stage 2 inserts the flow between the frozen feature and the prototype classifier. A velocity network transports each feature toward the prototype of its class, and the transported feature is classified by the same cosine rule as before. Two training objectives were compared. Standard flow matching samples $t \sim \mathcal{U}(0, 1)$, places $z_t = (1 - t)z + t \cdot p$ on the straight line from feature to prototype, and regresses the velocity onto $u = p - z$, backpropagating through one prediction. Rolled-out flow matching runs the full T -step Euler integration during training and supervises only the endpoint, $\|\hat{z}_T - p\|^2$, backpropagating through all T predictions; it matches training to inference at the cost of needing one model per T . Inference is identical for both. The grid was 3 combinations $\times$ 3 $K \times$ 3 seeds, one standard model and two rolled-out models per cell, 81 trained models in all.

Before training anything we reproduced Stage 1’s prototype baseline from the cached features rather than copying it: all 21 runs matched with $\max |\Delta| = 0.00\text{e}+00$ and the prototypes were byte-identical. The one design decision Stage 2 had to make was to L2-normalise features before the flow. Prototypes are unit-norm; raw features are not, so the target $u = p - z$ at $\|z\| \approx 30$ is very nearly just $-z$, with $\cos(u, -z) = 0.9995\text{--}0.9999$ on raw features and 0.22–0.35 after normalising. We then measured the decision rather than arguing it: training the same objective on raw features costs 15.8–20.3 pp, and on both ResNet-18 combinations the flow ends up worse than no flow at all.

Encoder / dataset	K	Baseline	std T=4	std T=12	roll T=4	roll T=12
ResNet-18/DTD	5	46.10 ± 1.89	44.47 ± 1.05	44.79 ± 1.01	40.99 ± 1.60	40.87 ± 1.46
	10	51.67 ± 1.33	50.11 ± 0.86	50.34 ± 0.66	44.77 ± 1.58	45.32 ± 1.23
	full	58.83 (1 run)	59.59 ± 0.61	59.56 ± 0.68	55.85 ± 0.47	55.82 ± 0.32
ResNet-18/Aircraft	5	16.46 ± 0.39	18.81 ± 1.01	18.94 ± 1.02	16.77 ± 0.84	17.03 ± 0.87
	10	20.19 ± 1.14	23.68 ± 0.39	23.91 ± 0.32	20.34 ± 0.53	20.09 ± 0.24
	full	25.47 (1 run)	31.04 ± 0.93	30.97 ± 0.56	25.64 ± 1.72	25.28 ± 1.69
DINOv2/Aircraft	5	23.79 ± 0.47	33.30 ± 0.68	32.82 ± 0.57	34.13 ± 0.59	33.86 ± 0.58
	10	27.74 ± 0.98	43.27 ± 1.09	42.48 ± 1.16	45.42 ± 1.10	45.35 ± 0.83
	full	34.26 (1 run)	56.52 ± 0.27	55.70 ± 0.31	52.05 ± 0.91	52.08 ± 0.82

Table 2. Stage 2 top-1 test accuracy (%). Bold-coloured: best FM cell in the row, shown only where it beats the baseline; three rows have none. 24 of 36 FM cells improve, from +22.26 pp down to −6.90 pp.

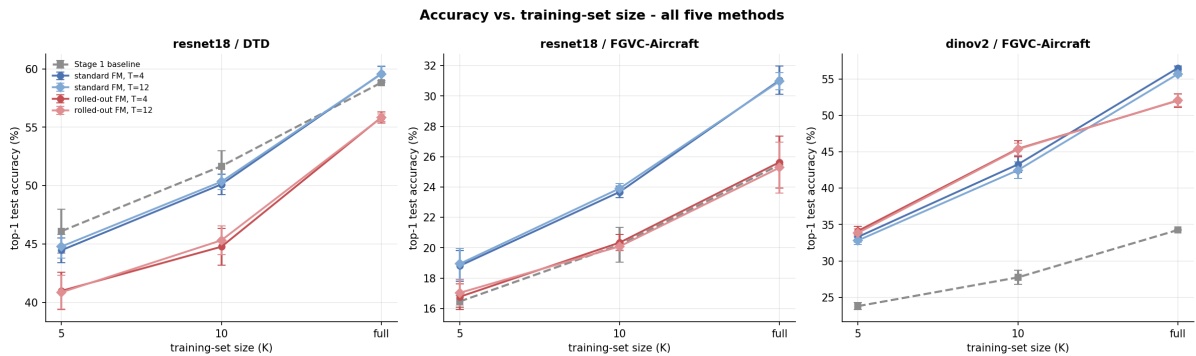


Figure 4. All five Stage 2 methods against training-set size, error bars over 3 seeds. FM is flat-to-harmful on DTD, clearly positive on ResNet-18/Aircraft, and transformative on DINOv2.

What predicts the gain

The gains range from −6.9 to +22.3 pp. Feature separability does not order them, and K orders them only within a combination. The quantity that orders all nine cells at once is the headroom column of Table 1, how much a trained linear probe had gained over the untrained prototype on that same cell. The nine cells rank identically by headroom and by gain (Spearman $\rho = 1.000$), the linear fit has $R^2 = 0.988$ and breaks even at 2.2 pp of headroom, and the only two cells below that line are exactly the two where FM loses.

READING THE RULE

The FM layer recovers roughly 70% of whatever training the classifier would have gained, minus a fixed cost of about 1.6 pp, while keeping the prototype structure and training no classifier at all. That also explains the DTD failures: there the prototype baseline is already level with the trained probe, so there is nothing for a transport to recover. Two caveats stand: nine cells is a small sample, and the linear probe is a reference point rather than a mechanism.

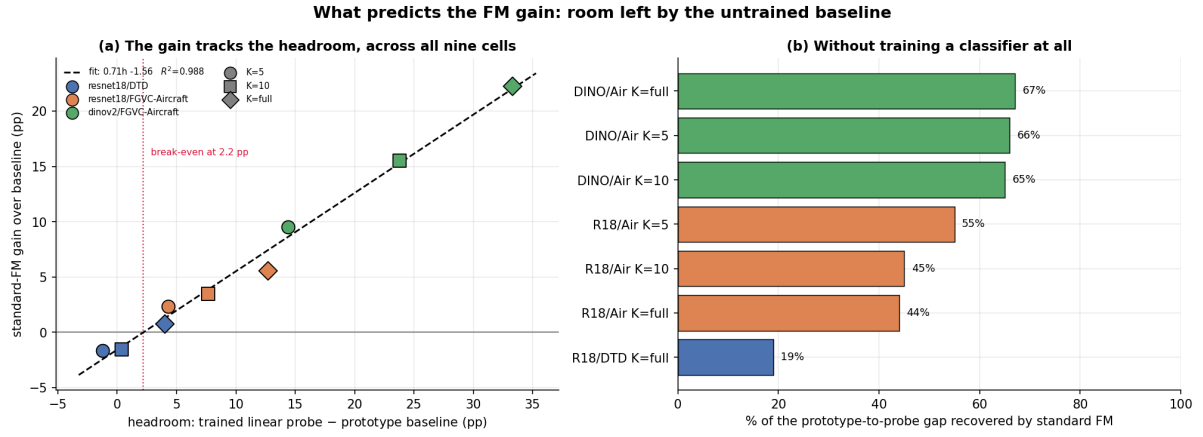


Figure 5. (a) Standard-FM gain over the prototype baseline against Stage 1 headroom, all nine cells, with the linear fit and its break-even point. (b) The fraction of the prototype-to-probe gap that standard FM recovers.

Why rolled-out training loses

Rolled-out FM was designed to remove the train/inference mismatch, and it does; the failure is not one of transport. It brings features closer to their target prototype than standard FM does, on training and test data alike, so it is not overfitting. The catch is what else moves: it raises similarity to the correct prototype and to the best competing prototype by nearly the same amount, and only the gap between the two decides the label. Against standard FM at the same T it wins 4 of 18 cells and loses up to 6.9 pp on DTD. Its trajectories also zigzag and loop, since only the endpoint is supervised: classifying the intermediate state at each Euler step shows accuracy collapsing mid-rollout, on ResNet-18/Aircraft to 9.2 pp below doing nothing, before recovering at the final step, and features travel up to 149% further from the target before returning. The number of Euler steps barely mattered for either objective, with under 1 pp between $T = 4$ and $T = 12$ everywhere. One of the 54 rolled-out runs diverged late in training; we added gradient-norm clipping at 1.0 to both objectives so they stay configuration-identical (it is inert for standard FM, ≤ 0.25 pp) and kept the unclipped run as evidence.

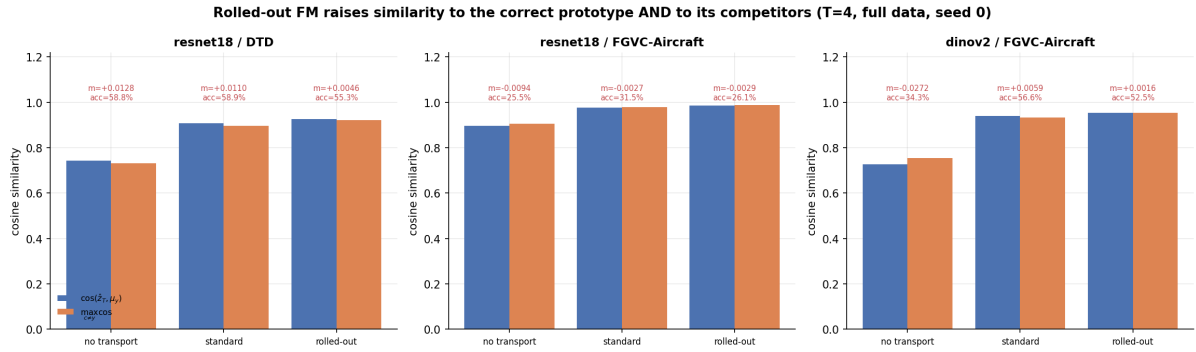


Figure 6. Cosine similarity to the correct prototype and to the best competing one after transport ($T = 4$, full data, seed 0). Both rise together under both objectives; the annotated margin is what decides the label, and standard FM's is wider everywhere.

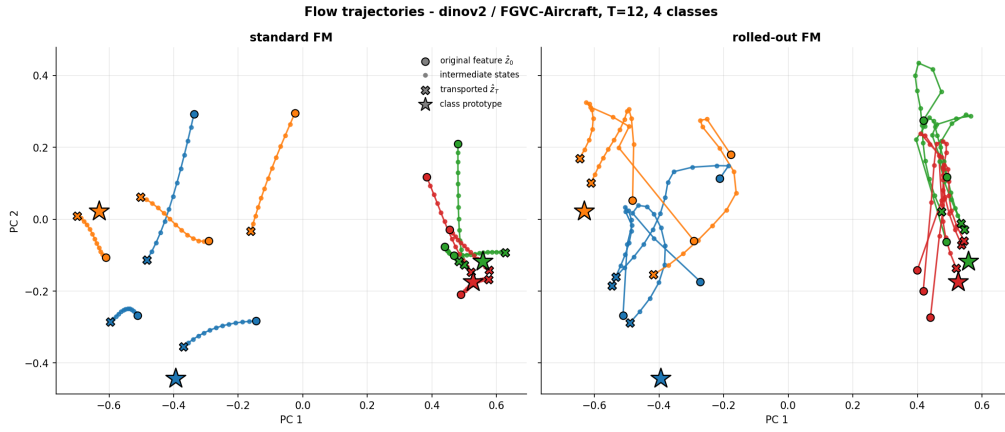


Figure 7. Flow trajectories under a joint PCA, DINOv2/Aircraft, $T = 12$, four classes: original feature (circle), intermediate states (dots), transported feature (cross), prototype (star). Standard FM takes near-straight paths; rolled-out FM loops.

4 Stage 3: flow matching before a frozen linear classifier STAGE 3

Stage 3 moves the flow in front of the linear probe from Stage 1, which is trained first and then frozen so that it never adapts to what the flow does. The flow is initialised to the exact identity (the velocity network’s readout is zeroed), so at epoch 0 the system is bit-for-bit the Stage 1 probe, $\Delta = 0.00$ pp with identical predictions on all 16,920 test images, and every result is caused by training the flow alone. Since Stage 1 had not saved W and b , Stage 3 re-derives the probes with its own training loop that imports Stage 1’s hyperparameter definition; the re-derived probes match Stage 1’s published accuracies at max $|\Delta| = 0.00$ pp, and an 11-test equivalence suite pins the two implementations to each other. We fixed $K = 10$ and $T = 4$, and ran the two core combinations plus ResNet-18/Aircraft, whose features Stage 1 measured as the most entangled of the nine and which is therefore the most informative test of whether a flow can rescue a poorly shaped representation.

Here the normalisation decision goes the other way. Stage 2 normalised because its decision rule, cosine similarity, is scale-invariant. Stage 3’s frozen classifier was trained on raw features, and an affine rule is not scale-invariant: $W(z/\|z\|) + b$ rescales Wz but leaves b untouched. Feeding the frozen probe normalised features costs 4.49, 5.34 and 3.43 pp on the three combinations, most on DINOv2 whose norms are already tightly clustered at 46.9–52.0, so even a near-uniform rescale is not harmless. The same fact is why the identity initialisation is load-bearing: a flow initialised the ordinary way would displace features on the first forward pass, and the classifier has no slack for that.

Two strategies, 225 runs

Strategy 1 is end-to-end: run the full rollout, push $\hat{z}$ through the frozen classifier, minimise $\text{CE}(W\hat{z} + b, y)$ by backpropagating through all four Euler steps, with two regularisers (displacement and velocity magnitude) swept at three strengths each. Strategy 2 is classifier-guided: roll z to $\hat{z}$, take a unit-normalised gradient step on $\hat{z}$ against the classifier’s loss to obtain a target $\hat{z}'$, detach it, and train the flow to transport $z \rightarrow \hat{z}'$ with a completely standard FM objective, refreshing targets every few epochs. The classifier says where to go; the FM machinery does the going. Strategy 1 ran 81 configurations-by-seeds and Strategy 2 ran 99.

The diagnostic that shaped both was measured at identity, before training. The frozen probe already classifies its own k-shot training set at 98.44–100.00%, with training cross-entropy as low as 0.005 against validation cross-entropy of 1.8–3.6. The objective both strategies chase is nearly exhausted before they start, so following it further is memorisation, and the real question is how little the flow can be allowed to move. For Strategy 1 the default learning rate of 10^{-3} , inherited from Stages 1 and 2, was the worst choice tested: it drives a 657k–788k-parameter network onto 470–1000 features so fast that validation accuracy never rises, and a tenfold smaller rate is worth about +1.1 pp, more than any regulariser. Constraining how fast the flow moves beat constraining how far. For Strategy 2 the guidance targets already sit at 100% training accuracy on two of three combinations, so the transform it learns is margin-increasing rather than error-correcting; refreshing targets every 10 epochs rather than every epoch was the best knob (+1.82 pp against +1.40 pp), normalising the guidance step mattered most of all (+0.16/+0.46 pp unnormalised against +1.4 to +1.8 normalised), and 91 of 99 runs improved on the probe.

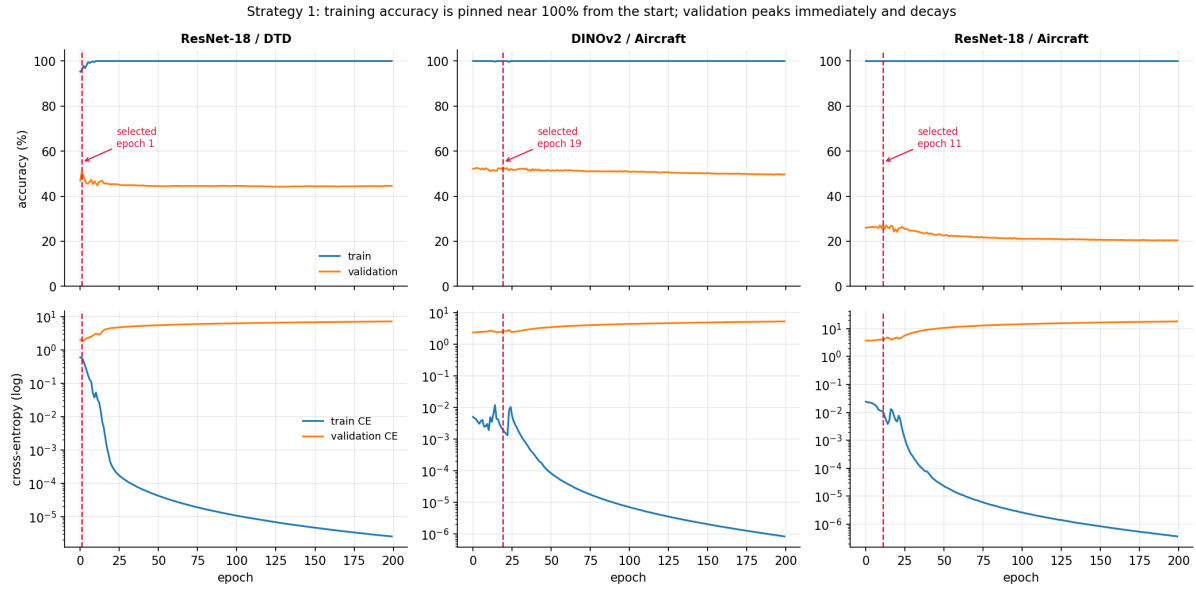


Figure 8. Strategy 1 training curves ($\text{lr } 10^{-3}$). Training accuracy starts at $\sim 100\%$ and stays there; training cross-entropy falls five orders of magnitude on data already classified correctly while validation cross-entropy rises from epoch 0. Checkpointing on validation accuracy, Stage 1’s own rule, is what makes the method work at all.

Combination	Probe	Strategy 1	ΔAcc	Strategy 2	ΔAcc
ResNet-18 / DTD	52.04%	52.15 ± 1.15	+0.11	53.24 ± 1.39	+1.21
DINOv2 / FGVC-Aircraft	51.54%	54.13 ± 1.13	+2.59	53.73 ± 1.53	+2.19
ResNet-18 / FGVC-Aircraft *	27.87%	28.97 ± 0.74	+1.10	29.92 ± 0.60	+2.05

Table 3. Stage 3 top-1 test accuracy, $K = 10$, $T = 4$, mean $\pm$ std over 3 seeds. Configurations were selected by mean validation accuracy, not test; validation and test independently picked the same ones ($\text{lr } 10^{-4}$ and refresh 10). * The additional combination beyond the two used elsewhere.

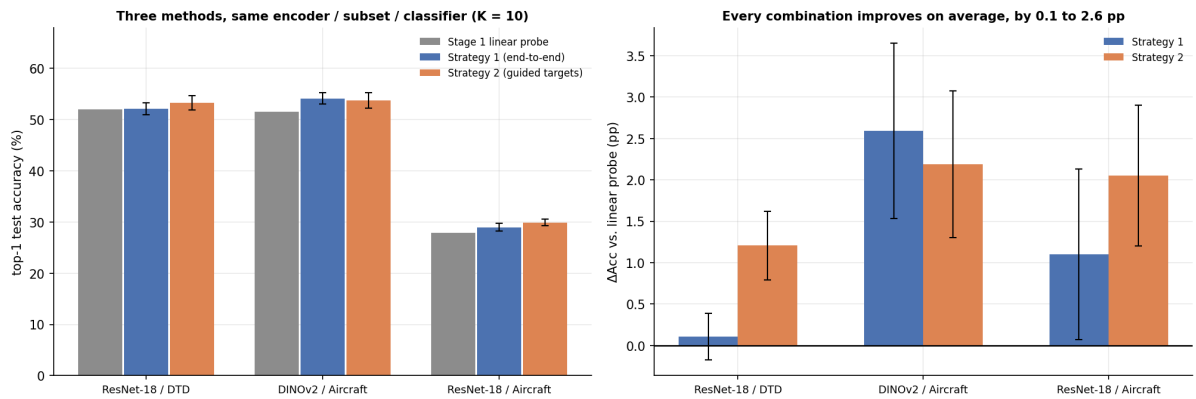


Figure 9. The three Stage 3 methods per dataset (left) and ΔAcc against the Stage 1 probe (right). Every combination improves under both objectives: Strategy 1 by +1.27 pp overall, Strategy 2 by +1.82 pp.

Held at a matched learning rate, the two strategies are indistinguishable on accuracy ($+0.13$ pp, $p = 0.80$, trading wins per combination; $+0.55$ pp at $p = 0.30$ even best-against-best). They differ, significantly, on robustness: at $\text{lr } 10^{-3}$ Strategy 2 leads by $+0.79$ pp ($p = 0.048$), because Strategy 1 optimises the classification loss directly and memorises labels within a handful of epochs (selected epochs 4/66/35 against 101/117/198), whereas Strategy 2 regresses onto a target vector through which the labels reach it only indirectly. Whether classifier-guided training beats end-to-end training is therefore answered no on accuracy and yes on robustness.

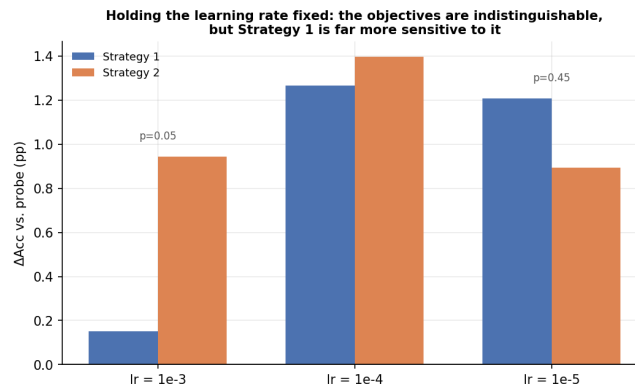


Figure 10. Objective versus tuning: the two strategies at each learning rate.

What the flow does to the geometry

Under a joint PCA on the same six test classes, Strategy 1 visibly stretches the whole space while Strategy 2’s panel is almost indistinguishable from the original, for an equal-or-better result: on DINOv2/Aircraft the mean displacement $\|\hat{z} - z\|$ is 34.94 under Strategy 1 and 4.60 under Strategy 2, roughly 8× less movement for +1.51 pp against +1.01 pp. Measured in the full feature space, the quantity that tracks the result is the classifier’s logit margin, the gap between the top logit and the runner-up: five of six strategy/combination cases widen it by 1.45–1.92×, and the single exception, Strategy 1 on DTD at 0.997×, is exactly the cell that delivered nothing on the full test set. Class separability, the training-free measure that predicted Stage 1, does not track the result and moves the wrong way where the gain is real (Strategy 1 on DINOv2/Aircraft loses 0.074 of separability while gaining accuracy). Strategy 2 needs so little movement because its target vector already points along the direction that widens the margin.

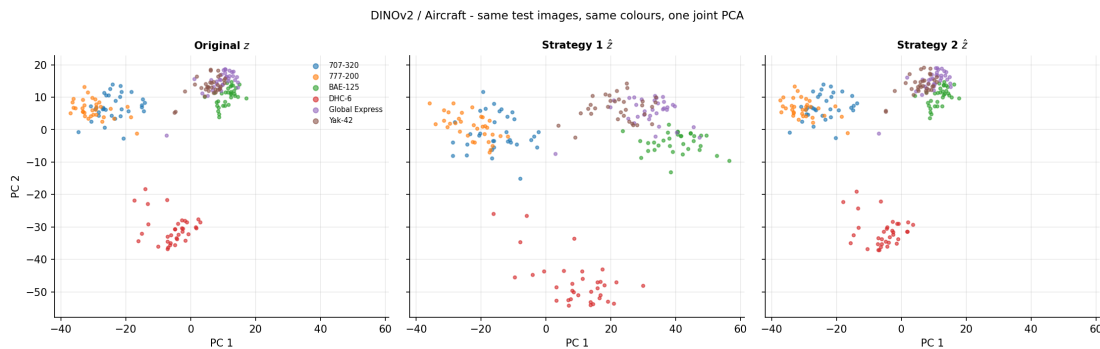


Figure 11. DINOv2/Aircraft test features under one joint PCA: original, after Strategy 1, after Strategy 2. Six classes, one seed; two components carry only 20.7–33.8% of the variance, which is why the geometry is also measured in the full space.

Finally, we unfroze the classifier and trained it jointly with the flow, alongside a classifier-only control that continues training the Stage 1 probe with no flow at all. The control moves the probe by +0.22 pp, which independently confirms that Stage 1 left it at its ceiling; the joint runs gain +1.24 pp, of which +1.02 pp is attributable to the flow; and unfreezing makes no measurable difference to the flow’s contribution (−0.02 pp, $p = 0.97$). The classifier’s learning rate can span 100× and its weight drift 50× with every result landing between +1.20 and +1.24 pp, and it drifts more when trained alone (0.104) than jointly with the flow (0.039). Keeping it frozen, which is what makes every other number in Stage 3 interpretable, costs nothing.

5 What the three stages say together

Read in sequence, the stages tell one story about where a flow-matching layer can and cannot add value, and a second about what a classifier actually consumes.

The headroom story

Stage 1 measured, for every cell, how much a trained linear classifier gains over an untrained prototype on the same frozen features (Table 1). Stage 2 found that this single quantity predicts the flow’s gain in front of the prototype classifier with $\rho = 1.000$: the flow recovers about 70% of the trained classifier’s advantage, without training a classifier, and loses only where there was nothing to recover. Stage 3 then spent that headroom on purpose by training the classifier first. The gain fell from up to +22.3 pp to +0.1 to +2.6 pp, and Stage 3’s own diagnostic shows why: the frozen probe already fits its k-shot training set at 96–100%, so the objective the flow chases is nearly exhausted before training starts. What remains, about a point, is whatever a nonlinear transform can add on top of an already-fitted linear boundary. The two placements are therefore not competing results but two points on one curve, and the curve was drawn in Stage 2 before Stage 3 was run.

Margin is what a classifier consumes

Each stage supplied one piece of the second lesson. In Stage 1, cluster structure in the raw features predicted which encoder would win, but a class-mean vector could not exploit fine-grained differences a trained boundary could. In Stage 2, rolled-out training pulled features closer to the right prototype than standard training did and still lost, because it pulled them closer to the wrong prototypes by the same amount; feature-space pictures of the +22 pp success case and the −1.6 pp failure case looked nearly identical. In Stage 3 the same pattern appeared from the other side: Stage 1’s separability measure moved the wrong way where the gain was real, and the logit margin was the only geometric quantity that tracked accuracy, flagging the one empty cell at 0.997×. Neither contraction toward a prototype nor tightness in a projection is a substitute for the decision gap, and a flow helps exactly when it widens that gap for the rule that follows it.

Smaller lessons that held across stages

- Whether to L2-normalise before the flow is decided by the decision rule, not by habit. Stage 2’s cosine rule required it (removing it cost 15.8–20.3 pp); Stage 3’s affine rule forbade it (adding it cost 3.4–5.3 pp). Both were measured rather than assumed.
- Backpropagating through the rollout is fragile. In Stage 2 it produced the only divergent run in 81 and needed gradient clipping; in Stage 3 the same objective memorised training labels within a few epochs at the default learning rate. The objectives that never differentiate through the rollout (standard FM in Stage 2, classifier-guided targets in Stage 3) were the robust ones each time.
- The number of Euler steps barely matters: under 1 pp between $T = 4$ and $T = 12$ throughout Stage 2, which is why Stage 3 fixed $T = 4$.
- Validation-accuracy checkpointing, Stage 1’s rule, was carried through unchanged and turned out to be load-bearing in Stage 3, where validation cross-entropy rises from epoch 0 and there is no regime in which training and validation improve together.

6 Limitations and reproducibility

Limitations, stated plainly

Three seeds give a t -test little power, so the p -values in Stage 3 are reported for completeness rather than leaned on; the more informative statement is how consistently the sign holds across full sweeps (83% and 92% of runs). Stage 2’s headroom rule rests on nine cells, and Stage 3’s three combinations cannot support a rule about which ones benefit, since DTD differs from the two Aircraft combinations in dataset, class count and training-set size at once. Stage 3 fixes one K and one T , so the K - and T -dependence measured in Stages 1 and 2 is not re-derived there. The Aircraft centre crop discards about 41% of image width and was left in place deliberately; it is a quantified limitation of the frozen encoder’s preprocessing rather than something we changed. And the weakest Stage 3 cell, Strategy 1 on DTD at +0.11 pp with one seed negative, is flagged independently by the margin diagnostic.

Reproducibility

Every number in the three stages was reproduced before we relied on it. Stage 1’s 27 probe runs match an independent rebuild of the pipeline within 0.26 pp and reproduce bit for bit on a clean re-run. Stage 2 re-derived Stage 1’s 21 prototype runs at max $|\Delta| = 0.00\text{e}+00$, retrained all 81 models from scratch with every accuracy and

loss curve identical, recomputed all 108 run accuracies from stored per-example predictions (max difference $6e-06$ pp), and asserts in its evaluation notebook that the results table matches its inputs. Stage 3 re-derives Stage 1's probes exactly, pins its training loop to Stage 1's with an 11-test equivalence suite, and passes 52 tests across Stages 2 and 3, including formula tests re-derived from each method's stated form; we validated the suite by breaking the implementation eight ways and, after fixing three tests that could not fail, confirmed all eight are caught. Every figure and table is generated by a tracked notebook backed by a tracked CSV, each step folder carries a manifest of what it wrote, and git confirms that no earlier stage's source was modified by a later one.

7 Conclusion

A flow-matching layer between a frozen encoder and a fixed classification rule is a cheap, classifier-free way to recover most of what training a classifier would have gained, and it is worth nothing more than that. In front of untrained prototypes it recovered about 70% of a trained probe's advantage, up to +22.3 pp; in front of the trained probe itself, with the headroom already spent, it added +0.1 to +2.6 pp. In both placements the flow helped exactly when it widened the decision margin of the rule behind it, and the training objectives that supervised the endpoint of a rollout, or differentiated through it, were consistently the less robust ones. The most useful outcome of the project is not any single accuracy but the fact that Stage 1's headroom column, a quantity available before a single flow is trained, told us in advance what the two later stages would find.